```
# visualization_utils.py
import os
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import ipywidgets as widgets
from IPython.display import display
from sklearn.metrics import r2_score

class BaseDashboard:
    def __init__(self, df_summary, is_test_run=False):
        self.df_summary = df_summary.copy()
        actual_candidates = ['Actual Diameter (km)', 'actual', 'Actual', 'Actual Size']
        for col in actual_candidates:
            if col in self.df_summary.columns and col != 'Actual Diameter (km)':
                self.df_summary = self.df_summary.rename(columns={col: 'Actual Diameter
(km)'})
                break
        self.is_test_run = is_test_run
        self.colors = {'Actual': '#000000', 'Actual Size': '#000000'}

    def build_metrics_html(self, df_active, pred_col):
        if df_active.empty or pred_col not in df_active.columns: return "<div style='color:
red;'>No data.</div>"
        act, pred = df_active['Actual Diameter (km)'].values, df_active[pred_col].values
        mae = np.mean(np.abs(act - pred))
        r2 = r2_score(act, pred) if len(df_active) > 1 else 1.0
        return f"""<div style='background-color: #f8f9fa; padding: 15px; border-radius: 6px;
font-family: sans-serif; color: #212529; border: 1px solid #dee2e6;'>
            <table style='width: 100%; text-align: center;'><tr style='color: #6c757d; font-
size: 13px;'><th>Sample Count</th><th>Mean Abs Error</th><th>R² Prediction</th></
tr>
            <tr style='font-size: 20px; font-weight: bold;'><td>{len(df_active)}</
td><td>{mae:.2f} km</td><td>{r2:.2f}</td></tr></table></div>"""

    def export_image(self, fig, filename_prefix):
        base_dir = 'figures/testing' if self.is_test_run else 'figures'
        os.makedirs(base_dir, exist_ok=True)
        full_path = os.path.join(base_dir, f"{filename_prefix}{'_test.png' if self.is_test_run
else '.png'}")
        try: fig.write_image(full_path, scale=3)
        except: pass

class GroupedBarChart(BaseDashboard):
    def render(self, scale, pred_col='Pred (km)', title='Asteroid Predictions Comparison'):
        if self.df_summary.empty: return self.build_metrics_html(self.df_summary,
pred_col), go.Figure()
```

```
        fig = go.Figure()
        sorted_df = self.df_summary.sort_values(by='Actual Diameter (km)',
ascending=False)
        unique_asteroids = sorted_df['Asteroid'].unique()
        actual_sizes = [sorted_df[sorted_df['Asteroid'] == a]['Actual Diameter
(km)'].values[0] for a in unique_asteroids if len(sorted_df[sorted_df['Asteroid'] == a]) > 0]

        fig.add_trace(go.Bar(y=unique_asteroids, x=actual_sizes, name='Actual Size',
orientation='h', marker=dict(color=self.colors.get('Actual'))))
        for eng in sorted(sorted_df['Engine'].unique()):
            sub = sorted_df[sorted_df['Engine'] == eng]
            fig.add_trace(go.Bar(y=sub['Asteroid'], x=sub[pred_col], name=eng,
orientation='h'))

        fig.update_layout(
            title=title, height=500, barmode='group',
            xaxis=dict(title=f"Diameter (km) ({scale})", type='log' if 'Log' in scale else 'linear'),
            yaxis=dict(title='Asteroid Name', autorange='reversed'),
            legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='left', x=0)
        )
        self.export_image(fig, filename_prefix=title.lower().replace(' ', '_'))
        return self.build_metrics_html(self.df_summary, pred_col), fig

class ConnectionLinePlot(BaseDashboard):
    def render(self, scale, pred_col='Pred (km)', title='Asteroid Predictions Error Delta'):
        if self.df_summary.empty: return self.build_metrics_html(self.df_summary,
pred_col), go.Figure()
        fig = go.Figure()
        for ast in self.df_summary['Asteroid'].unique():
            ast_df = self.df_summary[self.df_summary['Asteroid'] == ast]
            for eng in ast_df['Engine'].unique():
                row = ast_df[ast_df['Engine'] == eng]
                if not row.empty:
                    act_val, pred_val = row['Actual Diameter (km)'].values,
row[pred_col].values
                    fig.add_trace(go.Scatter(x=[ast, ast], y=[act_val, pred_val], mode='lines',
line=dict(color='#000000', width=1.5), showlegend=False, hoverinfo='skip'))

        unique_asteroids = sorted(self.df_summary['Asteroid'].unique())
        actual_sizes = [self.df_summary[self.df_summary['Asteroid'] == a]['Actual Diameter
(km)'].values[0] for a in unique_asteroids]
        fig.add_trace(go.Scatter(x=unique_asteroids, y=actual_sizes, name='Actual Size',
mode='markers', marker=dict(size=11, color=self.colors.get('Actual'), symbol='circle')))
        for eng in sorted(self.df_summary['Engine'].unique()):
            sub = self.df_summary[self.df_summary['Engine'] == eng]
            fig.add_trace(go.Scatter(x=sub['Asteroid'], y=sub[pred_col], name=eng,
mode='markers', marker=dict(size=9, symbol='circle-open', line=dict(width=1.5)),
```

```
hovertemplate="<b>%{x}</b><br>Engine: " + eng + f"<br>{pred_col}: %{{y:,.2f}}
km<extra></extra>"))

        fig.update_layout(
            title=title, height=480,
            yaxis=dict(title=f"Diameter (km) ({scale})", type='log' if 'Log' in scale else 'linear'),
            xaxis=dict(title='Asteroid Name'), legend=dict(orientation='h', yanchor='bottom',
y=1.02, xanchor='left', x=0)
        )
        self.export_image(fig, filename_prefix=title.lower().replace(' ', '_'))
        return self.build_metrics_html(self.df_summary, pred_col), fig

class EvaluationScatterPlot(BaseDashboard):
    def render(self, scale, pred_col='Pred (km)', title='Model Evaluation Scatter Plot'):
        if self.df_summary.empty: return self.build_metrics_html(self.df_summary,
pred_col), go.Figure()
        fig = go.Figure()
        for eng in sorted(self.df_summary['Engine'].unique()):
            sub = self.df_summary[self.df_summary['Engine'] == eng]
            x_vals = pd.to_numeric(sub['Actual Diameter (km)'])
            y_vals = pd.to_numeric(sub[pred_col])
            fig.add_trace(go.Scatter(x=x_vals, y=y_vals, mode='markers', name=eng,
text=sub['Asteroid'], marker=dict(size=12), hovertemplate="<b>%{text}</b><br>Engine:
" + eng + "<br>Actual: %{x} km<br>Pred: %{y} km<extra></extra>"))
        mn, mx = pd.to_numeric(self.df_summary['Actual Diameter (km)'].min()) * 0.8,
pd.to_numeric(self.df_summary['Actual Diameter (km)'].max()) * 1.2
        fig.add_trace(go.Scatter(x=[mn, mx], y=[mn, mx], mode='lines', name='Actual Size
(y = x)', line=dict(color='#000000', dash='dash'), hoverinfo='skip'))
        ax_t = 'log' if 'Log' in scale else 'linear'
        fig.update_layout(title=title, height=480, xaxis=dict(title='Actual Measured Diameter
(km)', type=ax_t), yaxis=dict(title='Predicted Model Diameter (km)', type=ax_t),
legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='left', x=0))
        self.export_image(fig, filename_prefix=title.lower().replace(' ', '_'))
        return self.build_metrics_html(self.df_summary, pred_col), fig

class FeatureScarcityPlot(BaseDashboard):
    def render(self, scale, pred_col='Pred (km)', title='H-Value Drop Impact'):
        engines = sorted(self.df_summary['Engine'].unique())
        drop_data = []
        for eng in engines:
            sub_df = self.df_summary[self.df_summary['Engine'] == eng]
            f_mape = sub_df[sub_df['Subset'] == 'full']['Error %'].mean()
            r_mape = sub_df[sub_df['Subset'] == 'restricted']['Error %'].mean()
            drop_data.append({"Engine": eng, "Full": f_mape, "Gap": r_mape - f_mape})
        df_drop = pd.DataFrame(drop_data).sort_values(by="Gap", ascending=True)
        fig = go.Figure()
        fig.add_trace(go.Bar(y=df_drop['Engine'], x=df_drop['Full'], name='Full Subset
```

```
(With H)', orientation='h', marker=dict(color='#2C3E50')))
        fig.add_trace(go.Bar(y=df_drop['Engine'], x=df_drop['Gap'], name='Error Delta (H-
Drop Impact)', orientation='h', marker=dict(color='#EF553B')))
        fig.update_layout(title=title, height=450, barmode='stack', template='plotly_white',
xaxis=dict(title='Mean Absolute Percentage Error (MAPE) %'), yaxis=dict(title='Engine'),
legend=dict(orientation='h', yanchor='bottom', y=1.02, xanchor='left', x=0))
        self.export_image(fig, filename_prefix=title.lower().replace(' ', '_'))
        return "<div>Stacked Informational Drop Analysis Rendered Successfully.</div>",
fig

def launch_interactive_plot(df_summary, plot_type='bar', is_test_run=False,
pred_col='Pred (km)', **kwargs):
    plot_factory = {
        'bar': GroupedBarChart,
        'lines': ConnectionLinePlot,
        'scatter': EvaluationScatterPlot,
        'scarcity': FeatureScarcityPlot
    }
    plot_class = plot_factory.get(plot_type.lower(), GroupedBarChart)
    w_sc = widgets.ToggleButtons(options=['Log Scale', 'Linear Scale'],
description='Scale:')
    w_met = widgets.HTML()
    def callback(scale):
        plotter = plot_class(df_summary, is_test_run)
        metrics_html, fig = plotter.render(scale, pred_col=pred_col, **kwargs)
        w_met.value = metrics_html
        fig.show()
    out = widgets.interactive_output(callback, {'scale': w_sc})
    display(w_met, out, widgets.HBox([w_sc], layout=widgets.Layout(padding='10px',
justify_content='center')))
```